NumPy Day 2: Shape and dimensions

All examples assume: `import numpy as np`

What I learned

Concept	Syntax	What it means
2D array (a table)	`np.array([[1, 2, 3], [4, 5, 6]])`	A list of lists. One inner list per row.
Shape	`arr.shape`	`(rows, columns)`, for example `(2, 3)`. A flat array shows `(6,)`.
Number of rows	`arr.shape[0]`	The first number in the shape pair.
Number of columns	`arr.shape[1]`	The second number in the shape pair.
Dimensions	`arr.ndim`	`1` for a flat array, `2` for a table.
Total elements	`arr.size`	All the numbers in the array (rows x columns).
Reshape	`arr.reshape(3, 2)`	Same numbers, new layout. Gives a new array, so save it with =.
Flatten	`arr.flatten()`	Unrolls a table, row by row, into one flat row. `ndim` goes from 2 to 1.
Reshape that doesn't fit	`arr.reshape(5, 3)`	On 12 numbers this gives a `ValueError`. Rows x columns must equal the size.

`shape`, `ndim` and `size` are data the array already holds, so they have no parentheses. `reshape()` and `flatten()` are actions, so they have parentheses.

Things to remember

1. Brackets decide flat or table. No inner brackets means flat. One inner pair `[ ]` per row means a table. `np.array([[1, 2, 3, 4, 5, 6]])` is a table with 1 row, not a flat array.
2. Array name first, then a dot, then the action. `seats.reshape(3, 4)` is right. `reshape.seats(3, 4)` and `flatten(classroom)` are wrong.
3. `reshape` and `flatten` give back a new array. Save it: `table = a.reshape(4, 3)`. The original stays the same.
4. Shape is always (rows, columns). `shape[0]` is rows and `shape[1]` is columns. The `0` and `1` are positions in the pair, not rows and columns themselves.
5. Rows x columns must equal the size. 12 numbers fit 3 x 4, but not 5 x 3.
6. No `( )` after an array's own name or its data. `arr([...])` gives `TypeError: 'numpy.ndarray' object is not callable`, and `table.shape([1])` fails too.
7. A `try` always needs an `except`.
8. Every file needs its own `import numpy as np` at the top, or you get `NameError: name 'np' is not defined`.
9. Put a label in front of a number with a comma: `print("Rows:", classroom.shape[0])`.

Practice problems and solutions

These are the problems from this day with working solutions and the real output. Try each one yourself before reading the solution.

Problem 1 (easy)

Make a 2D array called `scores` from `[[80, 90, 70], [60, 75, 85]]`. Print the array, then its `shape`, `ndim` and `size`.

SOLUTION

```
import numpy as np

scores = np.array([[80, 90, 70], [60, 75, 85]])
print(scores)
print(scores.shape)
print(scores.ndim)
print(scores.size)
```

OUTPUT

```
[[80 90 70]
 [60 75 85]]
(2, 3)
2
6
```

What it taught: `shape`, `ndim` and `size` describe a table without any parentheses. The shape is always rows first, then columns.

Problem 2 (easy)

Make a flat array `nums` from `[1, 2, 3, 4, 5, 6]` and print its `shape`. Reshape it into 2 rows and 3 columns, save it as `grid`, and print `grid`. Then print the `shape` of `grid`.

SOLUTION

```
import numpy as np

nums = np.array([1, 2, 3, 4, 5, 6])
print(nums.shape)

grid = nums.reshape(2, 3)    # 3 rows and 2 columns also works
print(grid)
print(grid.shape)
```

OUTPUT

```
(6,)
[[1 2 3]
 [4 5 6]]
(2, 3)
```

What it taught: `reshape` regroups the same numbers into a new rows x columns layout. The `(6,)` means one dimension holding 6 numbers.

Problem 3 (medium)

Make a flat array `data` from the numbers 1 to 12. Reshape it into 3 rows and 4 columns, save it as `table`, and print `table`, then its `shape`, `ndim` and `size`. Then, inside a `try`/`except ValueError`, run `data.reshape(5, 3)`. In the `except`, print `"That shape doesn't fit"`.

SOLUTION

```
import numpy as np

data = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
table = data.reshape(3, 4)
print(table)
print(table.shape)
print(table.ndim)
print(table.size)

try:
    print(data.reshape(5, 3))
except ValueError:
    print("That shape doesn't fit")
```

OUTPUT

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
(3, 4)
2
12
That shape doesn't fit
```

What it taught: `reshape` only works if rows x columns equals the number of elements. Otherwise NumPy raises a `ValueError`.

Problem 4 (medium)

A shop's sales for 12 months are in a flat array: `sales = np.array([120, 135, 150, 160, 155, 170, 180, 175, 190, 200, 210, 220])`. Reshape it into a table with 4 rows (quarters) and 3 columns (months), save it as `table`, and print it. Then print the number of quarters and the number of months per quarter, using `table.shape`.

SOLUTION

```
import numpy as np

sales = np.array([120, 135, 150, 160, 155, 170, 180, 175, 190, 200, 210, 220])
table = sales.reshape(4, 3)
print(table)

print("Quarters:", table.shape[0])
print("Months per quarter:", table.shape[1])
```

OUTPUT

```
[[120 135 150]
 [160 155 170]
 [180 175 190]
 [200 210 220]]
Quarters: 4
Months per quarter: 3
```

What it taught: `table.shape` is a pair of (rows, columns), and `shape[0]` and `shape[1]` pick out the row count and the column count.

Problem 5 (hard)

A teacher has marks for 2 students in 3 subjects: `marks = np.array([[70, 80, 90], [60, 65, 75]])`. Print `marks.shape`, then `marks.ndim`. Flatten `marks`, save it as `flat_marks`, and print it. Then print `flat_marks.shape` and `flat_marks.ndim`.

SOLUTION

```
import numpy as np

marks = np.array([[70, 80, 90], [60, 65, 75]])
print(marks.shape)
print(marks.ndim)

flat_marks = marks.flatten()
print(flat_marks)
print(flat_marks.shape)
print(flat_marks.ndim)
```

OUTPUT

```
(2, 3)
2
[70 80 90 60 65 75]
(6,)
1
```

What it taught: `flatten()` unrolls a table, row by row, into one flat row. The `ndim` drops from 2 to 1, and the original table stays unchanged.

Problem 6 (hard)

A classroom has 12 seats, numbered 1 to 12: `seats = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])`. Reshape it into 3 rows and 4 columns, save it as `classroom`, and print it. Print `Rows:` and the number of rows, `Columns:` and the number of columns, and `Total seats:` and the size. Then flatten `classroom` back into a flat array, save it as `back_to_flat`, and print it.

SOLUTION

```
import numpy as np

seats = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
classroom = seats.reshape(3, 4)
print(classroom)

print("Rows:", classroom.shape[0])
print("Columns:", classroom.shape[1])
print("Total seats:", classroom.size)

back_to_flat = classroom.flatten()
print(back_to_flat)
```

OUTPUT

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
Rows: 3
Columns: 4
Total seats: 12
[ 1  2  3  4  5  6  7  8  9 10 11 12]
```

What it taught: `shape[0]`, `shape[1]` and `size` describe the table, and `flatten()` undoes the reshape. The result matches the original `seats`.